Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Лабораторна робота №5

з дисципліни «Об'єктно-орієнтоване програмування»

на тему:

«Розробка багатовіконного інтерфейсу користувача для

графічного редактора об’єктів»

Виконала:

студентка групи ІМ-55

Горбань А.О

Номер в списку: **6**

Перевірив:

Рекечинський Д.О

Київ - 2026

Завдання:

1. Створити у середовищі розробки проект з ім’ям Lab5.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми. 4. Перевірити роботу програми. За потреби налагодити програму. 5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Виконання бонусних завдань:

- Бонус 1: Підсвічування обраного в таблиці об'єкта червоним пунктирним прямокутником на головному полотні.
- Бонус 2: Видалення виділеної в таблиці фігури з масиву об'єктів за допомогою клавіші DELETE.
- Бонус 3: Повний імпорт (завантаження) фігур із CSV-файлу з їхньої подальшою візуалізацією на полотні та в таблиці.

- **Архітектурне обмеження:** Заборонено створювати залежності модуля my_table від інших файлів проєкту (editor.h тощо). Взаємодія реалізується за допомогою паттерну **Listener (інтерфейс ITableListener)** у main.cpp та надсилання повідомлень WM_USER (PostMessage).

Варіант: 6

Студенти з парним номером (2, 4, 6, . . .) програмують об’єкт класу MyEditor на основі класичної реалізації Singleton.

Файл resource.h

```
#ifndef RESOURCE_H

#define RESOURCE_H




#define IDR_MAIN_MENU     101
```

```
#define IDB_TOOLBAR       102



#define IDM_POINT         201

#define IDM_LINE          202

#define IDM_RECTANGLE     203

#define IDM_ELLIPSE       204

#define IDM_LINEOO        205

#define IDM_CUBE          206

#define IDM_EXIT          207

#define IDM_ABOUT         208



#define IDM_TABLE         209

#define IDM_EXPORT        210

#define IDM_IMPORT        211



#define IDD_TABLE_DIALOG  301
```

```
#define IDC_LIST_TABLE    3001



#define WM_TABLE_SELECT_SHAPE (WM_USER + 100)

#define WM_TABLE_DELETE_SHAPE (WM_USER + 101)



#endif
```

Файл resource.rc

```
#include <windows.h>

#include "resource.h"


#pragma code_page(65001)


IDR_MAIN_MENU MENU

BEGIN

    POPUP "Файл"

    BEGIN

        MENUITEM "Вихід", IDM_EXIT

    END
```

```
    POPUP "Об'єкти"
    BEGIN
        MENUITEM "Крапка", IDM_POINT
        MENUITEM "Лінія", IDM_LINE
        MENUITEM "Прямокутник", IDM_RECTANGLE
        MENUITEM "Еліпс", IDM_ELLIPSE
        MENUITEM "Лінія з кружечками", IDM_LINEOO
        MENUITEM "Куб", IDM_CUBE
        MENUITEM SEPARATOR
        MENUITEM "Таблиця об'єктів", IDM_TABLE
        MENUITEM "Зберегти в CSV...", IDM_EXPORT
        MENUITEM "Завантажити з CSV...", IDM_IMPORT
    END
    POPUP "Довідка"
    BEGIN
        MENUITEM "Про програму", IDM_ABOUT
    END
END


IDB_TOOLBAR BITMAP "toolbar.bmp"
```

IDD_TABLE_DIALOG DIALOGEX 0, 0, 260, 180

STYLE DS_SETFONT | DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU

CAPTION "Таблиця об'єктів"

FONT 8, "MS Shell Dlg"

BEGIN

    CONTROL "", IDC_LIST_TABLE, "SysListView32", LVS_REPORT | LVS_SINGLESEL | WS_BORDER | WS_TABSTOP, 7, 7, 246, 166

END

Файл shape.h

```
#ifndef SHAPE_H

#define SHAPE_H



#include <windows.h>

#include <cmath>

#include <cwchar>

#include <wchar.h>



class Shape {

protected:

    long x1, y1, x2, y2;

public:

    virtual ~Shape() {}
```

```
    virtual void SetPoints(long startX, long startY, long endX, long endY) {
        x1 = startX; y1 = startY;
        x2 = endX;   y2 = endY;
    }

    virtual void Draw(HDC hdc) = 0;


    virtual void GetName(wchar_t* buf, int bufSize) const = 0;
    virtual void GetPoints(int& rx1, int& ry1, int& rx2, int& ry2) const {
        rx1 = (int)x1; ry1 = (int)y1;
        rx2 = (int)x2; ry2 = (int)y2;
    }
};

class PointShape : public virtual Shape {
public:
    void Draw(HDC hdc) override {
        SetPixel(hdc, x1, y1, RGB(0, 0, 0));
    }
    void GetName(wchar_t* buf, int bufSize) const override {
        wcsncpy(buf, L"Крапка", bufSize);
    }
};
```

```
class LineShape : public virtual Shape {
public:
    void Draw(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

        MoveToEx(hdc, x1, y1, NULL);
        LineTo(hdc, x2, y2);

        SelectObject(hdc, hOldPen);
        DeleteObject(hPen);
    }
    void GetName(wchar_t* buf, int bufSize) const override {
        wcsncpy(buf, L"Лінія", bufSize);
    }
};

class RectangleShape : public virtual Shape {
protected:
    virtual HBRUSH GetBrush() const {
        return CreateSolidBrush(RGB(144, 238, 144));
    }
public:
    void Draw(HDC hdc) override {
```

```
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HBRUSH hBrush = GetBrush();

        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        Rectangle(hdc, x1, y1, x2, y2);

        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
        if (hBrush != GetStockObject(NULL_BRUSH)) {
            DeleteObject(hBrush);
        }
    }
    void GetName(wchar_t* buf, int bufSize) const override {
        wcsncpy(buf, L"Прямокутник", bufSize);
    }
};

class EllipseShape : public virtual Shape {
public:
    void Draw(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HBRUSH hNullBrush = (HBRUSH)GetStockObject(NULL_BRUSH);
```

```
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hNullBrush);

        Ellipse(hdc, x1, y1, x2, y2);

        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
    }
    void GetName(wchar_t* buf, int bufSize) const override {
        wcsncpy(buf, L"Еліпс", bufSize);
    }
};

class LineOOShape : public LineShape, public EllipseShape {
public:
    void Draw(HDC hdc) override {
        LineShape::Draw(hdc);

        long ox1 = x1, oy1 = y1, ox2 = x2, oy2 = y2;
        int r = 5;

        SetPoints(ox1 - r, oy1 - r, ox1 + r, oy1 + r);
        EllipseShape::Draw(hdc);
```

```
        SetPoints(ox2 - r, oy2 - r, ox2 + r, oy2 + r);
        EllipseShape::Draw(hdc);

        SetPoints(ox1, oy1, ox2, oy2);
    }
    void GetName(wchar_t* buf, int bufSize) const override {
        wcsncpy(buf, L"Лінія з кружечками", bufSize);
    }
};

class CubeShape : public RectangleShape, public LineShape {
protected:
    HBRUSH GetBrush() const override {
        return (HBRUSH)GetStockObject(NULL_BRUSH);
    }
public:
    void Draw(HDC hdc) override {
        long ox1 = x1, oy1 = y1, ox2 = x2, oy2 = y2;

        int dx = (ox2 - ox1) / 3;
        int dy = (oy2 - oy1) / 3;

        // 1. Задня грань
        SetPoints(ox1 + dx, oy1 - dy, ox2 + dx, oy2 - dy);
```

```
        RectangleShape::Draw(hdc);


        // 2. Передня грань
        SetPoints(ox1, oy1, ox2, oy2);
        RectangleShape::Draw(hdc);


        // 3. Чотири ребра
        SetPoints(ox1, oy1, ox1 + dx, oy1 - dy);
        LineShape::Draw(hdc);


        SetPoints(ox2, oy1, ox2 + dx, oy1 - dy);
        LineShape::Draw(hdc);


        SetPoints(ox1, oy2, ox1 + dx, oy2 - dy);
        LineShape::Draw(hdc);


        SetPoints(ox2, oy2, ox2 + dx, oy2 - dy);
        LineShape::Draw(hdc);


        SetPoints(ox1, oy1, ox2, oy2);
    }
    void GetName(wchar_t* buf, int bufSize) const override {
        wcsncpy(buf, L"Куб", bufSize);
    }
};
```

```
#endif
```

Файл editor.h

```
#ifndef EDITOR_H

#define EDITOR_H



#ifndef _WIN32_IE

#define _WIN32_IE 0x0500

#endif



#include <windows.h>

#include <commctrl.h>

#include "shape.h"

#include "resource.h"



const int MAX_SHAPES = 106;



class MyEditor {

private:
```

```
    static MyEditor* pInstance;



    Shape* pcshape[MAX_SHAPES];

    int shapeCount;

    int currentTool;

    int selectedIndex;



    bool isDrawing;

    POINT startPoint;

    POINT currentPoint;



    HWND hWndParent;

    HWND hToolbar;

    HPEN hRubberPen;



    MyEditor();

    ~MyEditor();

    MyEditor(const MyEditor&) = delete;

    MyEditor& operator=(const MyEditor&) = delete;
```

```
    void UpdateWindowTitle();
    void DrawRubberBand(HWND hWnd, POINT pt1, POINT pt2);

public:
    static MyEditor* getInstance();
    static void destroyInstance();

    void Init(HWND hWnd);
    void OnSize();

    void OnCommand(WPARAM wParam);
    LRESULT OnNotify(LPARAM lParam);
    void OnLButtonDown(HWND hWnd, LPARAM lParam);
    void OnMouseMove(HWND hWnd, LPARAM lParam);
    void OnLButtonUp(HWND hWnd, LPARAM lParam);
    void OnPaint(HWND hWnd);

    int GetShapeCount() const { return shapeCount; }
```

```
    Shape* GetShape(int index) const {
        if (index >= 0 && index < shapeCount) return pcshape[index];
        return nullptr;
    }


    void SetSelectedIndex(int index) { selectedIndex = index; }
    void DeleteShape(int index);
    void ClearShapes();
    void AddShape(Shape* pShape);
};


#endif
```

Файл editor.cpp

```
#include "editor.h"


MyEditor* MyEditor::pInstance = nullptr;


MyEditor* MyEditor::getInstance() {
    if (pInstance == nullptr) {
```

```
        pInstance = new MyEditor();
    }
    return pInstance;
}

void MyEditor::destroyInstance() {
    if (pInstance != nullptr) {
        delete pInstance;
        pInstance = nullptr;
    }
}

MyEditor::MyEditor() {
    shapeCount = 0;
    selectedIndex = -1;
    currentTool = IDM_POINT;
    isDrawing = false;
    hToolbar = NULL;
    hWndParent = NULL;
```

```
    hRubberPen = NULL;



    for (int i = 0; i < MAX_SHAPES; ++i) {

        pcshape[i] = NULL;

    }

}



MyEditor::~MyEditor() {

    ClearShapes();

    if (hRubberPen) {

        DeleteObject(hRubberPen);

    }

}



void MyEditor::ClearShapes() {

    for (int i = 0; i < shapeCount; ++i) {

        if (pcshape[i]) {

            delete pcshape[i];

            pcshape[i] = NULL;
```

```
        }

    }

    shapeCount = 0;

    selectedIndex = -1;

}



void MyEditor::DeleteShape(int index) {

    if (index < 0 || index >= shapeCount) return;



    delete pcshape[index];

    for (int i = index; i < shapeCount - 1; ++i) {

        pcshape[i] = pcshape[i + 1];

    }

    pcshape[shapeCount - 1] = nullptr;

    shapeCount--;



    if (selectedIndex == index) selectedIndex = -1;

    else if (selectedIndex > index) selectedIndex--;

}
```

```
void MyEditor::AddShape(Shape* pShape) {
    if (shapeCount < MAX_SHAPES && pShape) {
        pcshape[shapeCount++] = pShape;
    }
}

void MyEditor::Init(HWND hWnd) {
    hWndParent = hWnd;

    INITCOMMONCONTROLSEX icex;
    icex.dwSize = sizeof(INITCOMMONCONTROLSEX);
    icex.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icex);

    hToolbar = CreateToolbarEx(
        hWnd,
        WS_CHILD | WS_VISIBLE | TBSTYLE_FLAT | TBSTYLE_TOOLTIPS,
        1, 6, GetModuleHandle(NULL), IDB_TOOLBAR,
```

```
        NULL, 0, 16, 16, 16, 16, sizeof(TBBUTTON)
    );

    if (hToolbar) {
        SendMessage(hToolbar, TB_BUTTONSTRUCTSIZE, (WPARAM)sizeof(TBBUTTON), 0);

        TBBUTTON tbb[6] = {
            { 0, IDM_POINT,     TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 },
            { 1, IDM_LINE,      TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 },
            { 2, IDM_RECTANGLE, TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 },
            { 3, IDM_ELLIPSE,   TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 },
            { 4, IDM_LINEOO,    TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 },
            { 5, IDM_CUBE,      TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 }
        };

        SendMessage(hToolbar, TB_ADDBUTTONS, 6, (LPARAM)&tbb);
        SendMessage(hToolbar, TB_AUTOSIZE, 0, 0);
        ShowWindow(hToolbar, SW_SHOW);
    }
```

```
    UpdateWindowTitle();
}

void MyEditor::OnSize() {
    if (hToolbar) {
        SendMessage(hToolbar, TB_AUTOSIZE, 0, 0);
    }
}

void MyEditor::UpdateWindowTitle() {
    const wchar_t* toolName = L"Крапка";
    switch (currentTool) {
        case IDM_POINT:     toolName = L"Крапка"; break;
        case IDM_LINE:      toolName = L"Лінія"; break;
        case IDM_RECTANGLE: toolName = L"Прямокутник"; break;
        case IDM_ELLIPSE:   toolName = L"Еліпс"; break;
        case IDM_LINEOO:    toolName = L"Лінія з кружечками"; break;
        case IDM_CUBE:      toolName = L"Куб"; break;
```

```
    }



    wchar_t title[128];

    wsprintf(title, L"Лабораторна 5 - [Поточний інструмент: %s]", toolName);

    SetWindowText(hWndParent, title);

}



void MyEditor::OnCommand(WPARAM wParam) {

    int id = LOWORD(wParam);

    if (id >= IDM_POINT && id <= IDM_CUBE) {

        currentTool = id;

        UpdateWindowTitle();

    } else if (id == IDM_ABOUT) {

        MessageBox(hWndParent, L"Лабораторна робота №5\nТема: Багатовіконний інтерфейс користувача", L"Про програму", MB_OK | MB_ICONINFORMATION);

    } else if (id == IDM_EXIT) {

        PostQuitMessage(0);

    }

}
```

```
LRESULT MyEditor::OnNotify(LPARAM lParam) {

    LPNMHDR pnmh = (LPNMHDR)lParam;

    if (pnmh->code == TTN_GETDISPINFO) {

        NMTTDISPINFOW* pttdi = (NMTTDISPINFOW*)lParam;

        switch (pttdi->hdr.idFrom) {

            case IDM_POINT:     pttdi->lpszText = (LPWSTR)L"Малювати крапку";
break;

            case IDM_LINE:      pttdi->lpszText = (LPWSTR)L"Малювати лінію";
break;

            case IDM_RECTANGLE: pttdi->lpszText = (LPWSTR)L"Малювати
прямокутник"; break;

            case IDM_ELLIPSE:   pttdi->lpszText = (LPWSTR)L"Малювати еліпс";
break;

            case IDM_LINEOO:    pttdi->lpszText = (LPWSTR)L"Малювати лінію з
кружечками"; break;

            case IDM_CUBE:      pttdi->lpszText = (LPWSTR)L"Малювати каркас
куба"; break;

        }

    }

    return 0;

}
```

```
void MyEditor::OnLButtonDown(HWND hWnd, LPARAM lParam) {

    isDrawing = true;

    startPoint.x = LOWORD(lParam);

    startPoint.y = HIWORD(lParam);

    currentPoint = startPoint;



    hRubberPen = CreatePen(PS_DOT, 1, RGB(0, 0, 0));



    SetCapture(hWnd);

}



void MyEditor::DrawRubberBand(HWND hWnd, POINT pt1, POINT pt2) {

    HDC hdc = GetDC(hWnd);

    int oldROP = SetROP2(hdc, R2_NOTXORPEN);

    HPEN hOldPen = (HPEN)SelectObject(hdc, hRubberPen);

    HBRUSH hNullBrush = (HBRUSH)GetStockObject(NULL_BRUSH);

    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hNullBrush);



    if (currentTool == IDM_LINE || currentTool == IDM_LINEOO) {
```

```
        MoveToEx(hdc, pt1.x, pt1.y, NULL);

        LineTo(hdc, pt2.x, pt2.y);

    } else if (currentTool == IDM_RECTANGLE || currentTool == IDM_CUBE) {

        Rectangle(hdc, pt1.x, pt1.y, pt2.x, pt2.y);

    } else if (currentTool == IDM_ELLIPSE) {

        Ellipse(hdc, pt1.x, pt1.y, pt2.x, pt2.y);

    }



    SelectObject(hdc, hOldBrush);

    SelectObject(hdc, hOldPen);

    SetROP2(hdc, oldROP);

    ReleaseDC(hWnd, hdc);

}



void MyEditor::OnMouseMove(HWND hWnd, LPARAM lParam) {

    if (!isDrawing || currentTool == IDM_POINT) return;



    DrawRubberBand(hWnd, startPoint, currentPoint);

    currentPoint.x = LOWORD(lParam);
```

```
    currentPoint.y = HIWORD(lParam);

    DrawRubberBand(hWnd, startPoint, currentPoint);

}



void MyEditor::OnLButtonUp(HWND hWnd, LPARAM lParam) {

    if (!isDrawing) return;



    if (currentTool != IDM_POINT) {

        DrawRubberBand(hWnd, startPoint, currentPoint);

    }



    ReleaseCapture();

    isDrawing = false;



    if (hRubberPen) {

        DeleteObject(hRubberPen);

        hRubberPen = NULL;

    }
```

```
if (shapeCount >= MAX_SHAPES) return;



POINT endPoint = { LOWORD(lParam), HIWORD(lParam) };

Shape* newShape = NULL;



switch (currentTool) {

    case IDM_POINT:     newShape = new PointShape(); break;

    case IDM_LINE:      newShape = new LineShape(); break;

    case IDM_RECTANGLE: newShape = new RectangleShape(); break;

    case IDM_ELLIPSE:   newShape = new EllipseShape(); break;

    case IDM_LINEOO:    newShape = new LineOOShape(); break;

    case IDM_CUBE:      newShape = new CubeShape(); break;

}



if (newShape) {

    newShape->SetPoints(startPoint.x, startPoint.y, endPoint.x, endPoint.y);

    pcshape[shapeCount++] = newShape;

    InvalidateRect(hWnd, NULL, TRUE);

}
```

```
}



void MyEditor::OnPaint(HWND hWnd) {

    PAINTSTRUCT ps;

    HDC hdc = BeginPaint(hWnd, &ps);



    // 1. Малюємо всі фігури

    for (int i = 0; i < shapeCount; ++i) {

        if (pcshape[i]) {

            pcshape[i]->Draw(hdc);

        }

    }



    // 2. Бонус 1: Підсвічуємо виділену фігуру червоним пунктиром

    if (selectedIndex >= 0 && selectedIndex < shapeCount &&
pcshape[selectedIndex]) {

        int x1 = 0, y1 = 0, x2 = 0, y2 = 0;

        pcshape[selectedIndex]->GetPoints(x1, y1, x2, y2);



        int minX = (x1 < x2 ? x1 : x2) - 5;
```

```
        int minY = (y1 < y2 ? y1 : y2) - 5;

        int maxX = (x1 > x2 ? x1 : x2) + 5;

        int maxY = (y1 > y2 ? y1 : y2) + 5;



        HPEN hRedPen = CreatePen(PS_DASH, 2, RGB(255, 0, 0));

        HBRUSH hNullBrush = (HBRUSH)GetStockObject(NULL_BRUSH);



        HPEN hOldPen = (HPEN)SelectObject(hdc, hRedPen);

        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hNullBrush);



        Rectangle(hdc, minX, minY, maxX, maxY);



        SelectObject(hdc, hOldPen);

        SelectObject(hdc, hOldBrush);

        DeleteObject(hRedPen);

    }



    EndPaint(hWnd, &ps);

}
```

Файл main.cpp

```
#include <windows.h>

#include <commctrl.h>

#include <string>

#include "editor.h"

#include "my_table.h"

#include "resource.h"



// Конкретна реалізація Listener у main.cpp

class MainTableListener : public ITableListener {

public:

    int OnTableGetShapeCount() override {

        return MyEditor::getInstance()->GetShapeCount();

    }



    bool OnTableGetShapeData(int index, wchar_t* nameBuf, int bufSize, int& x1,
int& y1, int& x2, int& y2) override {

        Shape* shape = MyEditor::getInstance()->GetShape(index);

        if (!shape) return false;

        shape->GetName(nameBuf, bufSize);
```

```
        shape->GetPoints(x1, y1, x2, y2);

        return true;

    }



    void OnTableClearShapes() override {

        MyEditor::getInstance()->ClearShapes();

    }



    void OnTableAddShape(const wchar_t* name, int x1, int y1, int x2, int y2)
override {

        Shape* newShape = nullptr;

        std::wstring nameStr(name);



        if (nameStr == L"Крапка" || nameStr == L"Точка") newShape = new
PointShape();

        else if (nameStr == L"Лінія") newShape = new LineShape();

        else if (nameStr == L"Прямокутник") newShape = new RectangleShape();

        else if (nameStr == L"Еліпс") newShape = new EllipseShape();

        else if (nameStr == L"Лінія з кружечками") newShape = new LineOOShape();

        else if (nameStr == L"Куб") newShape = new CubeShape();
```

```
        if (newShape) {
            newShape->SetPoints(x1, y1, x2, y2);
            MyEditor::getInstance()->AddShape(newShape);
        }
    }
};

static MainTableListener g_tableListener;

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    switch (message) {
        case WM_CREATE:
            MyEditor::getInstance()->Init(hWnd);
            MyTable::getInstance()->SetListener(&g_tableListener);
            break;

        case WM_SIZE:
            MyEditor::getInstance()->OnSize();
```

```
        break;



    case WM_COMMAND:

        if (LOWORD(wParam) == IDM_TABLE) {

            MyTable::getInstance()->Toggle(hWnd);

        } else if (LOWORD(wParam) == IDM_EXPORT) {

            MyTable::getInstance()->ExportToCSV(hWnd);

        } else if (LOWORD(wParam) == IDM_IMPORT) {

            MyTable::getInstance()->ImportFromCSV(hWnd);

        } else {

            MyEditor::getInstance()->OnCommand(wParam);

        }

        break;



    case WM_TABLE_SELECT_SHAPE:

        MyEditor::getInstance()->SetSelectedIndex((int)wParam);

        InvalidateRect(hWnd, NULL, TRUE);

        break;
```

```
    case WM_TABLE_DELETE_SHAPE:
        MyEditor::getInstance()->DeleteShape((int)wParam);
        MyTable::getInstance()->Update();
        InvalidateRect(hWnd, NULL, TRUE);
        break;


    case WM_NOTIFY:
        return MyEditor::getInstance()->OnNotify(lParam);


    case WM_LBUTTONDOWN:
        MyEditor::getInstance()->OnLButtonDown(hWnd, lParam);
        break;


    case WM_MOUSEMOVE:
        MyEditor::getInstance()->OnMouseMove(hWnd, lParam);
        break;


    case WM_LBUTTONUP:
        MyEditor::getInstance()->OnLButtonUp(hWnd, lParam);
```

```
            MyTable::getInstance()->Update();
            break;


        case WM_PAINT:
            MyEditor::getInstance()->OnPaint(hWnd);
            break;


        case WM_DESTROY:
            MyTable::destroyInstance();
            MyEditor::destroyInstance();
            PostQuitMessage(0);
            break;


        default:
            return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}
```

```
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR
lpCmdLine, int nCmdShow) {

    WNDCLASSEX wc = { sizeof(WNDCLASSEX) };

    wc.style         = CS_HREDRAW | CS_VREDRAW;

    wc.lpfnWndProc   = WndProc;

    wc.hInstance     = hInstance;

    wc.hCursor       = LoadCursor(NULL, IDC_ARROW);

    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

    wc.lpszMenuName  = MAKEINTRESOURCE(IDR_MAIN_MENU);

    wc.lpszClassName = L"Lab5WindowClass";



    RegisterClassEx(&wc);



    HWND hWnd = CreateWindowEx(

        0, L"Lab5WindowClass", L"Лабораторна 5",

        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT,

        800, 600, NULL, NULL, hInstance, NULL

    );



    ShowWindow(hWnd, nCmdShow);
```

```
    UpdateWindow(hWnd);



    MSG msg;

    while (GetMessage(&msg, NULL, 0, 0)) {

        HWND hTableDlg = MyTable::getInstance()->GetHWnd();

        if (!hTableDlg || !IsDialogMessage(hTableDlg, &msg)) {

            TranslateMessage(&msg);

            DispatchMessage(&msg);

        }

    }



    return (int)msg.wParam;

}
```

Файл my_table.h

```
#ifndef MY_TABLE_H

#define MY_TABLE_H



#ifndef _WIN32_IE

#define _WIN32_IE 0x0500
```

```
#endif



#include <windows.h>

#include <commctrl.h>



// Інтерфейс Слухача для зв'язку з main.cpp без жорстких залежностей

class ITableListener {

public:

    virtual ~ITableListener() {}

    virtual int OnTableGetShapeCount() = 0;

    virtual bool OnTableGetShapeData(int index, wchar_t* nameBuf, int bufSize,
int& x1, int& y1, int& x2, int& y2) = 0;

    virtual void OnTableClearShapes() = 0;

    virtual void OnTableAddShape(const wchar_t* name, int x1, int y1, int x2,
int y2) = 0;

};



class MyTable {

private:

    static MyTable* pInstance;
```

```
    HWND hDlg;

    HWND hListView;

    ITableListener* pListener; // Вказівник на Listener



    MyTable();

    ~MyTable();

    MyTable(const MyTable&) = delete;

    MyTable& operator=(const MyTable&) = delete;



    static INT_PTR CALLBACK DlgProc(HWND hDlg, UINT uMsg, WPARAM wParam, LPARAM
lParam);

    void InitListView();



public:

    static MyTable* getInstance();

    static void destroyInstance();



    void SetListener(ITableListener* listener) { pListener = listener; }



    void Toggle(HWND hWndParent);
```

```
    void Update();

    void ExportToCSV(HWND hWndParent);

    void ImportFromCSV(HWND hWndParent);



    HWND GetHWnd() const { return hDlg; }

};



#endif
```

Файл my_table.cpp

```
#include "my_table.h"

#include "resource.h"

#include <fstream>

#include <sstream>

#include <string>



MyTable* MyTable::pInstance = nullptr;



MyTable* MyTable::getInstance() {

    if (!pInstance) {
```

```
        pInstance = new MyTable();

    }

    return pInstance;

}



void MyTable::destroyInstance() {

    if (pInstance) {

        delete pInstance;

        pInstance = nullptr;

    }

}



MyTable::MyTable() {

    hDlg = NULL;

    hListView = NULL;

    pListener = nullptr;

}



MyTable::~MyTable() {
```

```
    if (hDlg) {
        DestroyWindow(hDlg);
        hDlg = NULL;
    }
}

void MyTable::Toggle(HWND hWndParent) {
    if (!hDlg) {
        hDlg = CreateDialogParam(
            GetModuleHandle(NULL),
            MAKEINTRESOURCE(IDD_TABLE_DIALOG),
            hWndParent,
            DlgProc,
            (LPARAM)this
        );
        ShowWindow(hDlg, SW_SHOW);
    } else {
        if (IsWindowVisible(hDlg)) {
            ShowWindow(hDlg, SW_HIDE);
```

```
        } else {

            ShowWindow(hDlg, SW_SHOW);

            Update();

        }

    }

}



INT_PTR CALLBACK MyTable::DlgProc(HWND hDlg, UINT uMsg, WPARAM wParam, LPARAM
lParam) {

    switch (uMsg) {

        case WM_INITDIALOG: {

            MyTable* pTable = (MyTable*)lParam;

            SetWindowLongPtr(hDlg, GWLP_USERDATA, (LONG_PTR)pTable);

            pTable->hDlg = hDlg;

            pTable->hListView = GetDlgItem(hDlg, IDC_LIST_TABLE);

            pTable->InitListView();

            pTable->Update();

            return TRUE;

        }
```

```
        case WM_NOTIFY: {

            LPNMHDR pnmh = (LPNMHDR)lParam;

            if (pnmh->idFrom == IDC_LIST_TABLE) {

                if (pnmh->code == LVN_ITEMCHANGED) {

                    LPNMLISTVIEW pnmv = (LPNMLISTVIEW)lParam;

                    if ((pnmv->uNewState & LVIS_SELECTED) != (pnmv->uOldState &
LVIS_SELECTED)) {

                        if (pnmv->uNewState & LVIS_SELECTED) {

                            PostMessage(GetParent(hDlg), WM_TABLE_SELECT_SHAPE,
(WPARAM)pnmv->iItem, 0);

                        } else {

                            PostMessage(GetParent(hDlg), WM_TABLE_SELECT_SHAPE,
(WPARAM)-1, 0);

                        }

                    }

                }

                else if (pnmh->code == LVN_KEYDOWN) {

                    LPNMLVKEYDOWN pnkd = (LPNMLVKEYDOWN)lParam;

                    if (pnkd->wVKey == VK_DELETE) {

                        MyTable* pTable = (MyTable*)GetWindowLongPtr(hDlg,
GWLP_USERDATA);

                        if (pTable && pTable->hListView) {
```

```
                            int sel = ListView_GetNextItem(pTable->hListView,
-1, LVNI_SELECTED);

                            if (sel != -1) {

                                PostMessage(GetParent(hDlg),
WM_TABLE_DELETE_SHAPE, (WPARAM)sel, 0);

                            }

                        }

                    }

                }

            return TRUE;

        }



        case WM_COMMAND:

            if (LOWORD(wParam) == IDCANCEL || LOWORD(wParam) == IDOK) {

                ShowWindow(hDlg, SW_HIDE);

                return TRUE;

            }

            break;
```

```
        case WM_CLOSE:
            ShowWindow(hDlg, SW_HIDE);
            return TRUE;

        case WM_NCDESTROY:
            return TRUE;
    }
    return FALSE;
}

void MyTable::InitListView() {
    if (!hListView) return;

    ListView_SetExtendedListViewStyle(hListView, LVS_EX_FULLROWSELECT |
LVS_EX_GRIDLINES);

    LVCOLUMN lvc;
    lvc.mask = LVCF_TEXT | LVCF_WIDTH | LVCF_SUBITEM;

    const wchar_t* headers[] = { L"Назва", L"x1", L"y1", L"x2", L"y2" };
```

```
    int widths[] = { 140, 60, 60, 60, 60 };



    for (int i = 0; i < 5; ++i) {

        lvc.iSubItem = i;

        lvc.pszText = (LPWSTR)headers[i];

        lvc.cx = widths[i];

        ListView_InsertColumn(hListView, i, &lvc);

    }

}



void MyTable::Update() {

    if (!hDlg || !IsWindowVisible(hDlg) || !hListView || !pListener) return;



    ListView_DeleteAllItems(hListView);



    int count = pListener->OnTableGetShapeCount();



    for (int i = 0; i < count; ++i) {

        wchar_t nameBuf[64] = L"Фігура";
```

```
        int x1 = 0, y1 = 0, x2 = 0, y2 = 0;



        if (!pListener->OnTableGetShapeData(i, nameBuf, 64, x1, y1, x2, y2))
continue;



        LVITEM lvi = { 0 };

        lvi.mask = LVIF_TEXT;

        lvi.iItem = i;

        lvi.pszText = nameBuf;

        ListView_InsertItem(hListView, &lvi);



        wchar_t strBuf[32];



        wsprintf(strBuf, L"%d", x1);

        ListView_SetItemText(hListView, i, 1, strBuf);



        wsprintf(strBuf, L"%d", y1);

        ListView_SetItemText(hListView, i, 2, strBuf);



        wsprintf(strBuf, L"%d", x2);
```

```
        ListView_SetItemText(hListView, i, 3, strBuf);



        wsprintf(strBuf, L"%d", y2);

        ListView_SetItemText(hListView, i, 4, strBuf);

    }

}



void MyTable::ExportToCSV(HWND hWndParent) {

    if (!pListener) return;



    wchar_t szFile[260] = L"shapes.csv";



    OPENFILENAME ofn;

    ZeroMemory(&ofn, sizeof(ofn));

    ofn.lStructSize = sizeof(ofn);

    ofn.hwndOwner = hWndParent;

    ofn.lpstrFile = szFile;

    ofn.nMaxFile = sizeof(szFile) / sizeof(wchar_t);

    ofn.lpstrFilter = L"CSV файли (*.csv)\0*.csv\0Усі файли (*.*)\0*.*\0";
```

```
    ofn.nFilterIndex = 1;
    ofn.Flags = OFN_PATHMUSTEXIST | OFN_OVERWRITEPROMPT;


    if (GetSaveFileName(&ofn) == TRUE) {
        char szFileA[260];
        WideCharToMultiByte(CP_ACP, 0, szFile, -1, szFileA, 260, NULL, NULL);


        std::wofstream outFile(szFileA);
        if (outFile.is_open()) {
            outFile << L"Назва,x1,y1,x2,y2\n";


            int count = pListener->OnTableGetShapeCount();


            for (int i = 0; i < count; ++i) {
                wchar_t nameBuf[64] = L"";
                int x1 = 0, y1 = 0, x2 = 0, y2 = 0;


                if (pListener->OnTableGetShapeData(i, nameBuf, 64, x1, y1, x2,
y2)) {
                    outFile << nameBuf << L"," << x1 << L"," << y1 << L"," <<
```

```
x2 << L"," << y2 << L"\n";

                }

            }

            outFile.close();

            MessageBox(hWndParent, L"Дані успішно збережено в CSV файл!",
L"Експорт", MB_OK | MB_ICONINFORMATION);

        }

    }

}



void MyTable::ImportFromCSV(HWND hWndParent) {

    if (!pListener) return;



    wchar_t szFile[260] = L"";



    OPENFILENAME ofn;

    ZeroMemory(&ofn, sizeof(ofn));

    ofn.lStructSize = sizeof(ofn);

    ofn.hwndOwner = hWndParent;

    ofn.lpstrFile = szFile;
```

```
ofn.nMaxFile = sizeof(szFile) / sizeof(wchar_t);
ofn.lpstrFilter = L"CSV файли (*.csv)\0*.csv\0Усі файли (*.*)\0*.*\0";
ofn.nFilterIndex = 1;
ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;



if (GetOpenFileName(&ofn) == TRUE) {
    char szFileA[260];
    WideCharToMultiByte(CP_ACP, 0, szFile, -1, szFileA, 260, NULL, NULL);



    std::wifstream inFile(szFileA);
    if (inFile.is_open()) {
        pListener->OnTableClearShapes();



        std::wstring line;
        bool firstLine = true;



        while (std::getline(inFile, line)) {
            if (line.empty()) continue;
            if (firstLine && line.find(L"Назва") != std::wstring::npos) {
```

```
            firstLine = false;

            continue;

        }

        firstLine = false;



        std::wstringstream ss(line);

        std::wstring nameStr, x1Str, y1Str, x2Str, y2Str;



        if (std::getline(ss, nameStr, L',') &&

            std::getline(ss, x1Str, L',') &&

            std::getline(ss, y1Str, L',') &&

            std::getline(ss, x2Str, L',') &&

            std::getline(ss, y2Str, L',')) {



            int x1 = _wtoi(x1Str.c_str());

            int y1 = _wtoi(y1Str.c_str());

            int x2 = _wtoi(x2Str.c_str());

            int y2 = _wtoi(y2Str.c_str());
```

```
                    pListener->OnTableAddShape(nameStr.c_str(), x1, y1, x2,
y2);

                }

            }

            inFile.close();



            Update();

            InvalidateRect(hWndParent, NULL, TRUE);

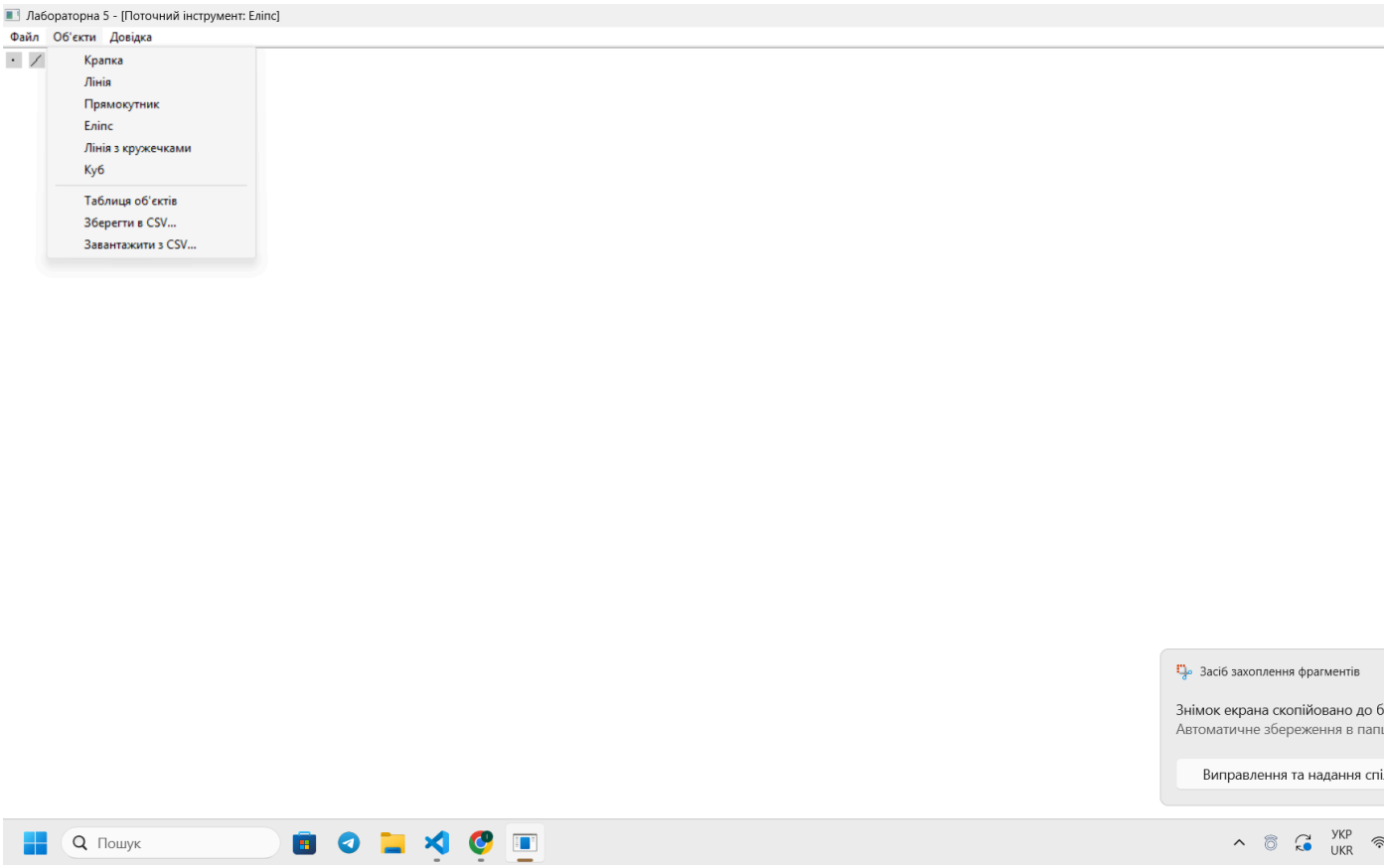
            MessageBox(hWndParent, L"Дані успішно завантажено з CSV!",
L"Імпорт", MB_OK | MB_ICONINFORMATION);

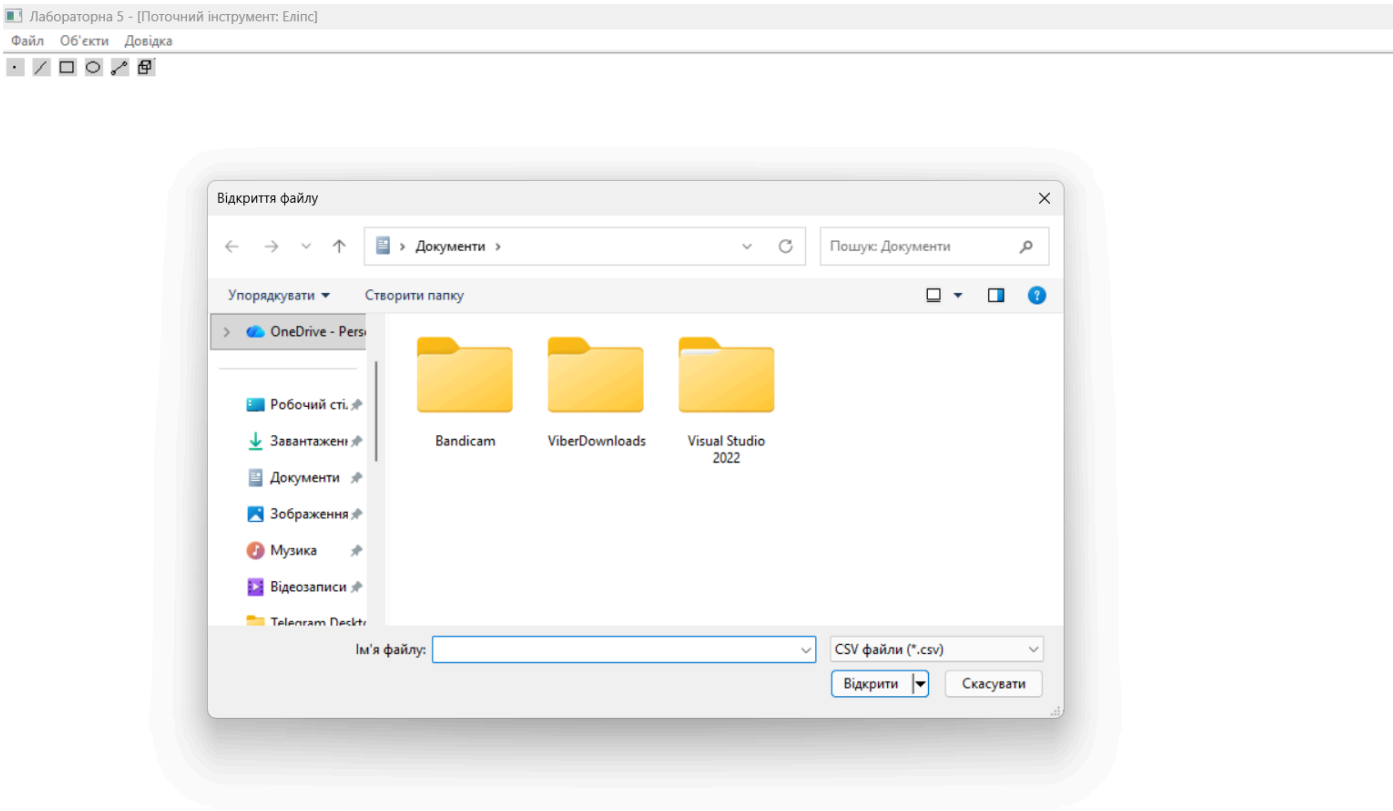
        }

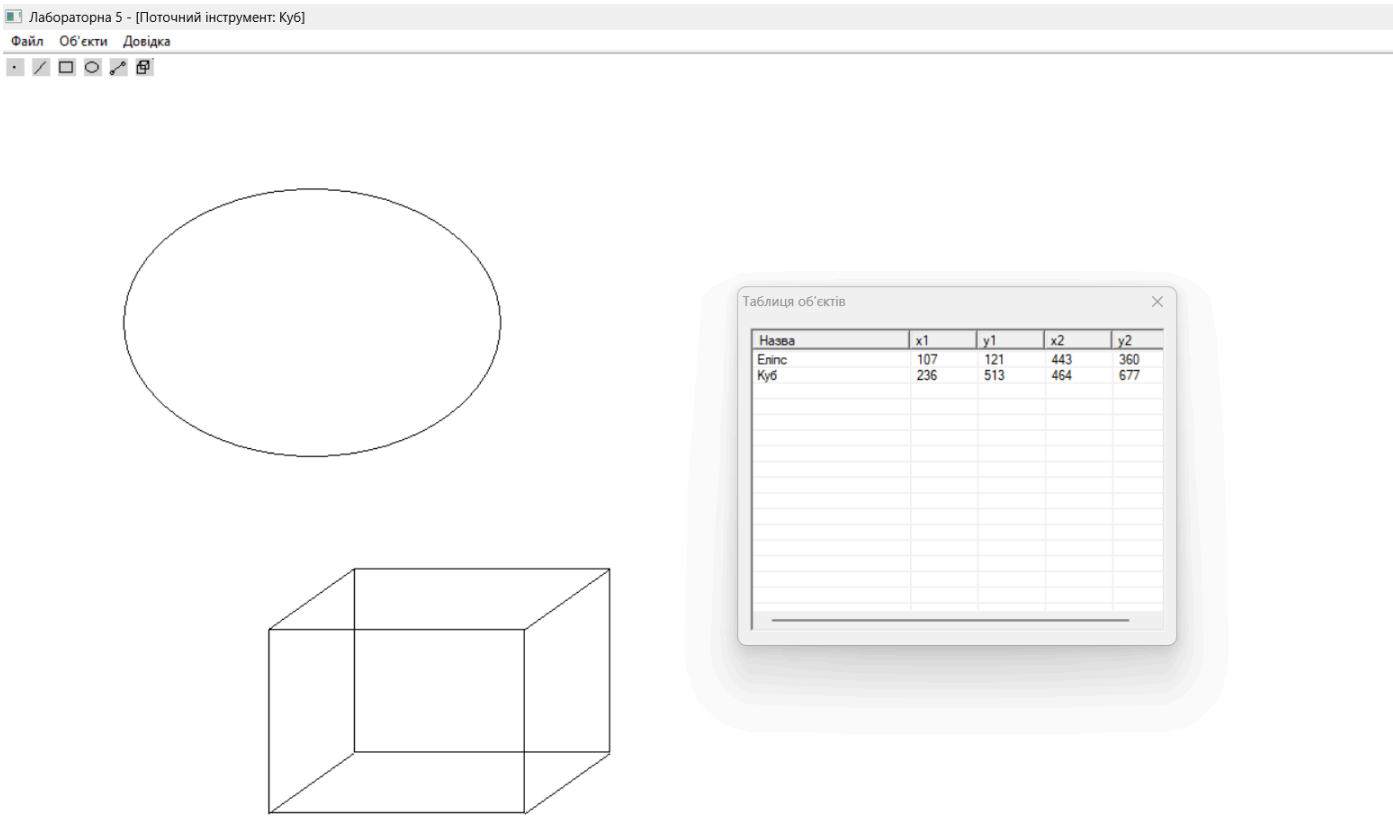
    }

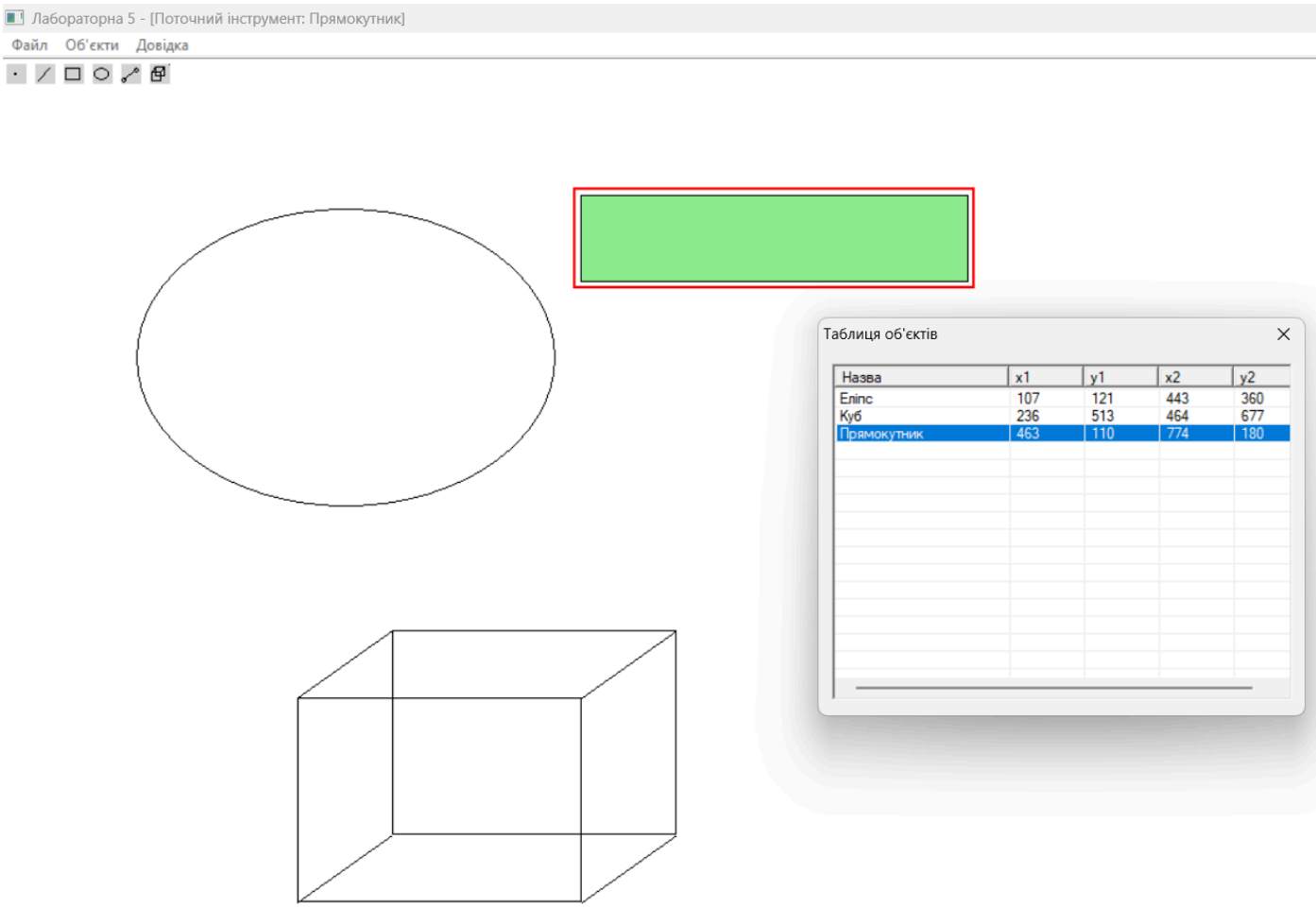
}
```

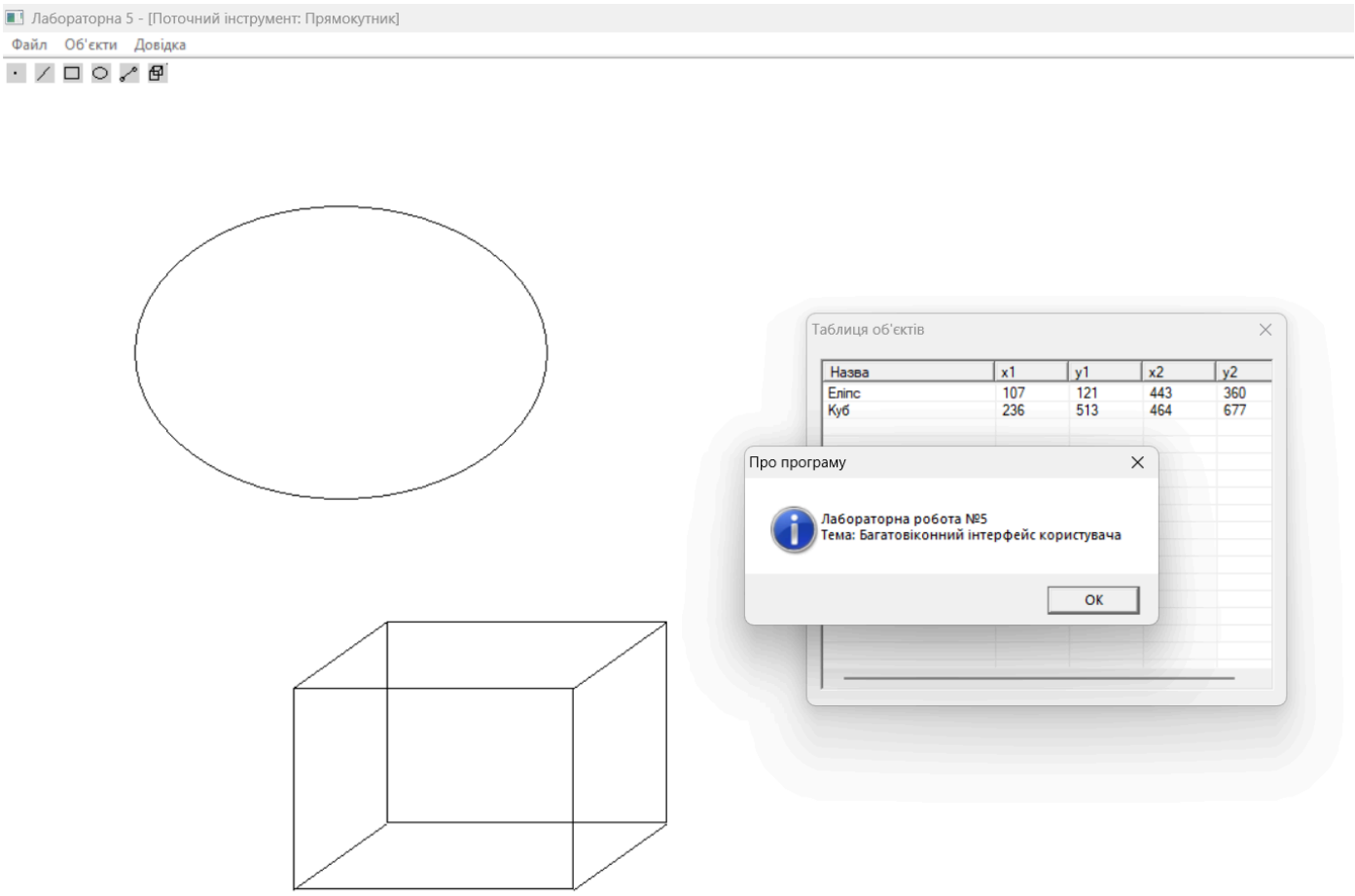
Скріншоти:


Лабораторна 5 - [Поточний інструмент: Еліпс]
Файл
Об'єкти
Довідка
Крапка
Лінія
Прямокутник
Еліпс
Лінія з кружечками
Куб
Таблиця об'єктів
Зберегти в CSV...
Завантажити з CSV...
Засіб захоплення фрагментів
Знімок екрана скопійовано до б
Автоматичне збереження в папц
Виправлення та надання спі
Пошук
УКР
UKR

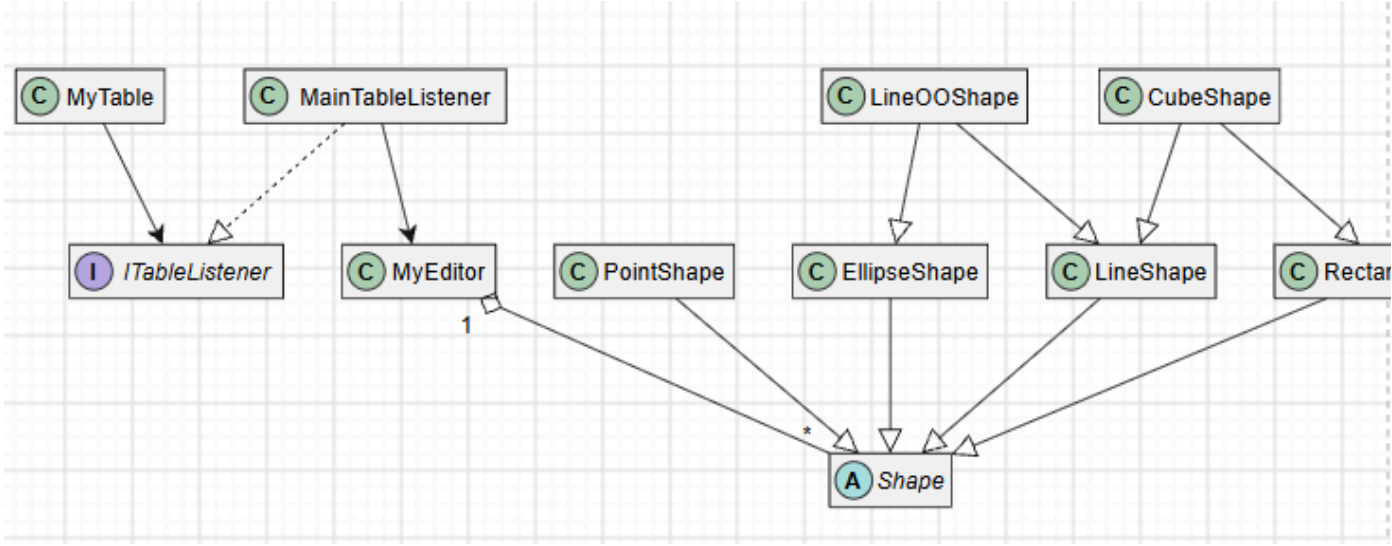

Лабораторна 5 - [Поточний інструмент: Еліпс]
Файл
Об'єкти
Довідка
Відкриття файлу
Документи
Пошук: Документи
Упорядкувати
Створити папку
OneDrive - Pers
Робочий сті
Завантажені
Документи
Зображення
Музика
Відеозаписи
Bandicam
ViberDownloads
Visual Studio 2022
Ім'я файлу:
CSV файли (*.csv)
Відкрити
Скасувати


Лабораторна 5 - [Поточний інструмент: Куб]
Файл
Об'єкти
Довідка
Таблиця об'єктів
Назва x1 y1 x2 y2
Еліпс 107 121 443 360
Куб 236 513 464 677


Лабораторна 5 - [Поточний інструмент: Прямокутник]
Файл
Об'єкти
Довідка
Таблиця об'єктів
Назва x1 y1 x2 y2
Еліпс 107 121 443 360
Куб 236 513 464 677
Прямокутник 463 110 774 180



Діаграма класів:



Висновки:

У ході виконання лабораторної роботи №5 було успішно застосовано паттерн Singleton для централізованого керування станом редактора та таблиці. Реалізовано немодальне діалогове вікно WinAPI (SysListView32). Для забезпечення слабозв'язаної архітектури та усунення прямої залежності модуля таблиці від редактора було впроваджено паттерн **Listener (ITableListener)**, а двосторонню

синхронізацію подій забезпечено через Windows-повідомлення WM_USER. Усі додаткові бонусні завдання (підсвічування об'єктів, видалення клавішею DELETE та повноцінний CSV імпорт/експорт) виконано в повному обсязі.